PRD: MCP-type Guardrails Service for LLM/RAG

- **문서 버전**: v1.0
- **작성일**: 2026-05-19
- **작성자**: bclim@edentns.com
- **상태**: Draft
- **참고 자료**: `.doc/Perplexity-Guardrails for LLM,RAG, 도메인별.md`

---

1. 개요 (Overview)

1.1 배경

LLM/RAG 시스템이 엔터프라이즈·규제 도메인으로 확산되면서, 입력·출력·행동 단계 전반에 안전·정책·품질을 강제하는 **가드레일(Guardrails)** 계층이 필수 요소가 되었다. 가드레일은 (1) Prompt Injection·Jailbreak 차단, (2) PII·민감정보 유출 방지, (3) 도메인·규제 위반 응답 차단, (4) 에이전트 행동 제한을 수행한다.

본 서비스는 가드레일 정책 결정·탐지·감사 기능을 **MCP(Model Context Protocol) Server**로 패키징하여, Claude Desktop·Gemini CLI/Desktop·Cursor 등 MCP-compatible 에이전트가 표준 인터페이스로 호출할 수 있도록 제공한다.

1.2 목표

- LLM/RAG 에이전트가 input/output을 본 호출 전후에 가드레일 MCP로 검증하여 PII·도메인·규제 위반을 차단한다.
- 도메인별로 별도 MCP 서버를 배포하지 않고, **단일 서비스에서 도메인 키 기반 정책 번들**을 동적으로 로드/적용한다.
- 한국·영어·일본어 등 다국어 정책을 동일 인프라에서 운영한다.
- 정책·탐지 결정을 PostgreSQL에 감사 로그로 영속화하고, Redis 캐시로 저지연 응답을 보장한다.

1.3 비목표 (Non-goals)

- LLM 모델 자체 학습/파인튜닝
- 자체 RAG 검색 엔진 구현 (외부 RAG의 입력·출력 검증만 담당)
- End-user UI (관리자 콘솔은 포함, 일반 사용자용 채팅 UI는 제외)
- 신원 인증/SSO 자체 구현 (외부 IdP 위임)

1.4 핵심 차별 포인트

1. **도메인 키 기반 멀티테넌시**: 클라이언트 설정 단계에서 `domains: ["manufacturing-food"]` 또는 `domains: ["manufacturing-food", "manufacturing-chemical"]` 같이 배열로 지정하면, 동일 MCP 서버 인스턴스가 해당 도메인 번들만 로드하여 판정한다.
2. **다국어 동시 지원**: `locale` 파라미터로 `ko-KR`, `en`, `ja-JP` 등 혼합 입력도 정규화 후 일관된 정책 평가.
3. **Privacy-first**: GDPR/PIPA/HIPAA 등 규제 태그를 모든 규칙에 매핑하여 규제 영향 분석을 지원.
4. **Hot reload 정책**: JSON/YAML 정책 파일을 Git/DB에서 무중단 reload.

---

2. 핵심 사용자 및 사용 사례

2.1 사용자(Actor)

Actor	설명
LLM Agent Developer	Claude Desktop/Gemini/Cursor 등 MCP 클라이언트 개발자
Domain Compliance Officer	도메인별 정책(JSON) 작성·승인
Platform Operator	MCP 서버 운영·모니터링·감사 로그 점검
End User (간접)	에이전트 UX를 통해 가드레일 결과(block/redact 메시지)를 수신

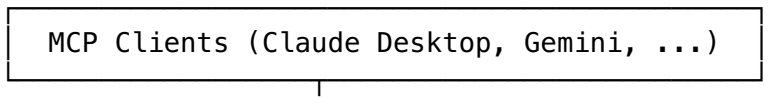
2.2 대표 사용 사례

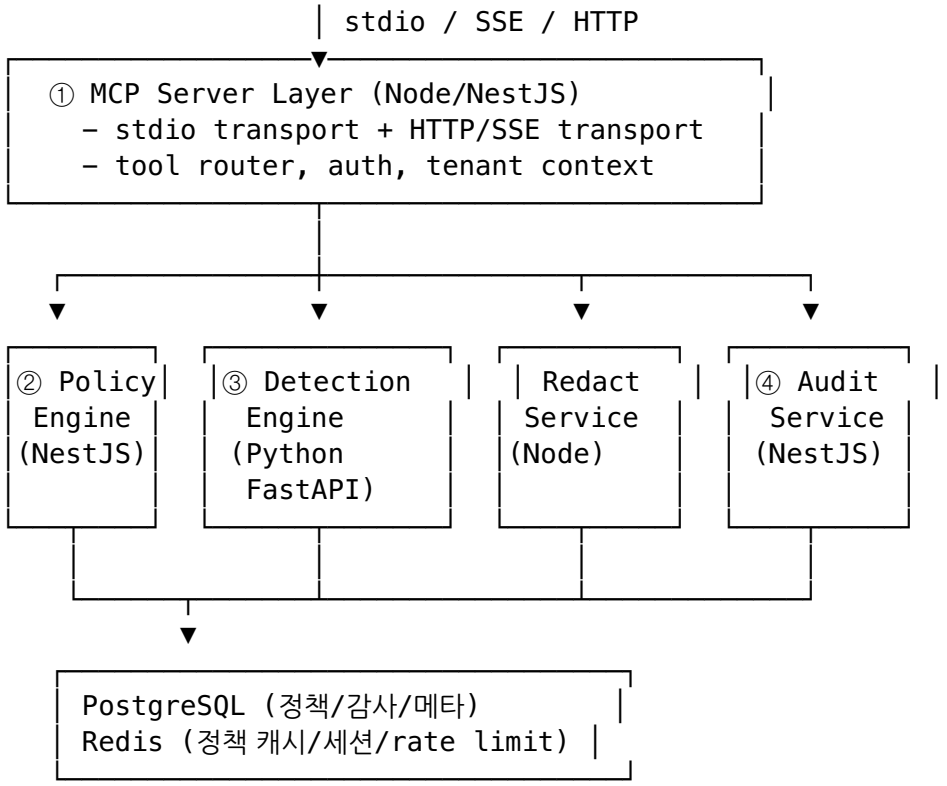
1. **식품 제조 챗봇 (Manufacturing.Food)**
   - 작업자가 "포장라인 반장 홍길동(010-1234-5678) 연락처 기록해줘" 입력 → `validate_input` → block + redact
2. **금융 RAG 어시스턴트 (Finance.RetailBanking)**
   - 모델 응답에 SSN 포함 → `validate_output` → redact
3. **다중 도메인 챗봇**
   - 한 조직에서 `manufacturing-food` + `manufacturing-chemical` 동시 적용 → 식품·화학 규제 동시 평가
4. **RAG 인덱싱 전처리**
   - RAG 인덱스 빌드 시 문서별로 `sanitize_input` 호출하여 PII 가명화

---

3. 시스템 아키텍처

3.1 4-계층 구조





3.2 컴포넌트별 책임

① MCP Server Layer (Node + TypeScript + NestJS)

- MCP 표준 프로토콜 핸들러 (stdio + HTTP/SSE 듀얼 transport)
- Tool 목록: `validate_input`, `sanitize_input`, `validate_output`, `redact_output`, `classify_privacy`, `get_policy_bundle`, `explain_decision`, `health_check`
- 클라이언트 인증 (API Key/OAuth)
- 테넌트 컨텍스트(`tenantId`, `domains[]`, `locale`) 결정
- 내부 마이크로서비스(Policy/Detection/Audit) 호출 오케스트레이션

② Policy Engine (NestJS)

- JSON 정책 번들 CRUD (Domain × Locale × IO_type)
- Bundle hot reload (PostgreSQL → Redis 캐시 갱신)
- Whitelist/Exception 관리
- 정책 버전 관리 (semver + 변경 이력)
- 도메인 키 매핑 (`manufacturing-food` → bundle resolver)

③ Detection Engine (Python + FastAPI)

- ko-KR/en/ja-JP 혼합 입력 정규화 (NFKC, 공백·구분자, 토큰화)
- 정규식 매처 (RRN, SSN, 마이넘버, 카드, 전화, 이메일, IBAN 등)
- 키워드/사전 매처 (병명·약물·정당명·종교 등 도메인 사전)
- NER (PERSON, ORG, LOCATION, DISEASE, MED_DRUG) — spaCy/ transformers/Presidio
- ML Classifier (LLM-as-judge 옵션)
- Luhn check, 체크섬 검증

- Risk score aggregation

Redact Service (Node)

- 패턴별 마스킹 전략 (****, [REDACTED], [PERSON])
- Generalization (시·구 단위, 역할 기반)
- Aggregation (개별 → 통계 변환)

④ Audit Service (NestJS)

- 모든 판정 결과 PostgreSQL 저장
- 원문은 해시(SHA-256) + 메타데이터만 보존 (옵션: 원문 암호화 저장)
- 감사 로그 조회 API
- Anomaly 탐지 (특정 세션 반복 high-risk 등)

3.3 데이터 저장소

PostgreSQL (필수)

용도: - `policies`, `policy_versions`, `whitelists`, `bundles` - `audit_logs`, `decision_events` - `tenants`, `domain_registry`, `api_keys` - `test_cases`, `test_runs`

Redis (필수)

용도: - 정책 번들 캐시 (`bundle:{domain}:{locale}:{io_type}`) - 탐지 결과 캐시 (입력 해시 → 결정) - Rate limiting (per tenant / per API key) - 세션 컨텍스트 (multi-turn risk score tracking) - Distributed lock (정책 reload 동시성 제어)

---

4. 도메인 확장성 설계 (핵심 요구사항)

4.1 단일 MCP, 다중 도메인 모델

기존 안: 도메인별로 별도 MCP 서버 배포 → 클라이언트 설정 폭증, 운영 부담.

채택 안: 단일 MCP 서버 인스턴스 + 클라이언트 측 `domains` 키 배열.

4.1.1 클라이언트 설정 예 (Claude Desktop)

```
{
  "mcpServers": {
    "guardrails": {
      "command": "node",
      "args": ["/opt/guardrails/dist/server.js"],
      "env": {
        "GUARDRAILS_API_KEY": "key_xxx",
        "GUARDRAILS_TENANT": "acme-corp",
        "GUARDRAILS_DOMAINS": "manufacturing-food",
```

```
        "GUARDRAILS_LOCALES": "ko-KR,en",
        "GUARDRAILS_ENDPOINT": "https://guardrails.example.com"
      }
    }
  }
}
```

복수 도메인 예 (식품 + 화학 제조사):

```
{
  "env": {
    "GUARDRAILS_DOMAINS": "manufacturing-food,manufacturing-
        chemical",
    "GUARDRAILS_LOCALES": "ko-KR,en,ja-JP"
  }
}
```

4.1.2 도메인 키 명명 규칙

```
<industry>-<sub_industry>[-<function>]
```

예: - `manufacturing-food` - `manufacturing-chemical` - `manufacturing-automotive-battery` - `finance-retail-banking` - `finance-investment` - `healthcare-clinical` - `healthcare-pharma-rd` - `public-gov-tax` - `retail-ecommerce`

4.1.3 도메인 레지스트리 스키마

```
interface DomainRegistry {
  domainKey: string;          // "manufacturing-food"
  parentKey?: string;         // "manufacturing" (상위 상속)
  displayName: { [locale: string]: string };
  description: string;
  defaultLocales: string[];   // ["ko-KR", "en"]
  policyBundleIds: string[];  // bundle id 참조
  regulationTags: string[];   // ["KR_PIPA", "EU_GDPR", "FDA"]
  status: 'active' | 'beta' | 'deprecated';
  inheritsFrom: string[];     // 상위 도메인 정책 상속 ["global"]
  version: string;            // semver
}
```

4.1.4 정책 평가 시 도메인 머지 순서

요청에 `domains: ["manufacturing-food", "manufacturing-chemical"]`이 들어오면 다음 순서로 정책을 합친다.

1. `global` (모든 도메인 공통 PII/안전 정책)
2. `manufacturing` (제조 공통)
3. `manufacturing-food` (식품 특화)
4. `manufacturing-chemical` (화학 특화)
5. `tenant-override` (테넌트 커스텀)

충돌 시 **더 엄격한(restrictive) 규칙 우선** 원칙을 적용한다 (block > redact > warn > allow).

4.2 새 도메인 추가 워크플로우

1. Compliance Officer가 관리 콘솔에서 `domainKey` 등록
2. 정책 JSON 작성 (input/output × locale)
3. Whitelist/예외 정의
4. 테스트 케이스 작성 (최소 20개)
5. Staging 환경에서 회귀 테스트
6. 정책 버전 승격 (draft → active)
7. Redis 캐시 무효화 → 클라이언트가 다음 호출부터 자동 적용

---

5. 정책 데이터 모델

5.1 Rule JSON Schema

```
interface GuardrailRule {
  ruleId: string;                    // "MF_FOOD_PRIV_KR_IN_001"
  domain: string;                    // "manufacturing-food"
  locale: string;                    // "ko-KR" | "en" | "ja-JP" |
        "*"
  ioType: 'input' | 'output' | 'both';
  category: 'pii' | 'safety' | 'compliance' | 'ip' | 'fraud'
          | 'prompt_injection' | 'scope' | 'hallucination'
          | 'sensitive' | 'children' | 'biometrics' |
        'credentials'
          | 'location' | 'health' | 'financial';
  subCategory: string;               // "employee_contact"
  description: string;
  patternType: 'regex' | 'keyword' | 'classifier'
             | 'llm_classifier' | 'regex+classifier'
             | 'generator_policy';
  patterns?: {
    regex?: string[];
    keywords?: string[];
    entities?: string[];             // ["PERSON", "PHONE"]
    luhnCheck?: boolean;
    minContextWindow?: number;
    requiresPersonLink?: boolean;
  };
  severity: 'low' | 'medium' | 'high' | 'critical';
  priority: number;                  // 0-1000 (높을수록 우선)
  action: 'allow' | 'warn' | 'redact' | 'sanitize'
        | 'block' | 'block_or_redact' | 'block_or_pseudonymize'
        | 'aggregate_only' | 'refuse' | 'escalate'
        | 'add_disclaimer' | 'route';
  messageTemplate: { [locale: string]: string };
  regulationTags: string[];          // ["KR_PIPA", "EU_GDPR_Art9"]
  examplesPositive: string[];        // 매치되어야 하는 예시
```

```
  examplesNegative: string[];      // 매치되면 안 되는 예시
  enabled: boolean;
  version: string;
  createdAt: string;
  updatedAt: string;
}
```

5.2 Bundle 구조

```
policies/
  global/
    privacy/
      input.ko-KR.json
      input.en.json
      input.ja-JP.json
      output.ko-KR.json
      output.en.json
      output.ja-JP.json
  manufacturing/
    safety/
      input.ko-KR.json
      ...
  manufacturing-food/
    privacy/
      input.ko-KR.json
      output.ko-KR.json
      input.en.json
      output.en.json
    compliance/
      input.ko-KR.json     # 식품위생법, 표시광고
      ...
    ip/
      input.ko-KR.json     # 영업비밀/레시피
  manufacturing-chemical/
    safety/
      input.ko-KR.json     # MSDS, 위험물
    compliance/
      ...
  bundles/
    manufacturing-food.bundle.json
    manufacturing-chemical.bundle.json
```

5.3 Whitelist Schema

```
interface Whitelist {
  id: string;
  domain: string;
  locale: string;
  category: 'regulation_term' | 'generic_process_term'
          | 'aggregated_expression' | 'role_expression';
  patterns: string[];
  description: string;
```

```
  scoreAdjustment: number;  // -0.1 ~ -0.5 (오탐 감소용)
}
```

5.4 카테고리별 기본 룰 수 목표 (도메인 무관 Privacy)

카테고리	룰 수	비고
식별정보 (IDs)	30	RRN, SSN, 마이넘버, 여권 등
연락처 (Contact)	30	이메일·전화·주소
금융 (Financial)	30	카드·계좌·결제
계정/인증 (Credentials)	30	비밀번호·OTP·API key
건강 (Health)	30	진단·약물·환자기록
위치 (Location)	20	주소·GPS
생체 (Biometrics)	10	지문·얼굴 임베딩
민감속성 (Sensitive)	20	인종·종교·정치
아동 (Children)	20	미성년자
로그/메타 (Log)	10	원본 로그
기타 (재식별 등)	10	linkage attack
합계	**240**	(도메인 무관, locale별 추가 확장)

도메인 특화 룰은 도메인별 별도 +200 ~ +500 수준으로 확장.

---

6. MCP Tool 인터페이스 명세

6.1 `validate_input`

입력:

```
{
  "text": "포장라인 반장 홍길동(010-1234-5678)에게 연락해",
  "domain": "manufacturing-food",
  "locale": "ko-KR",
  "userRole": "operator",
  "channel": "chat",
  "sessionId": "sess_abc123"
}
```

출력:

```
{
  "decision": "block",
  "riskScore": 0.94,
  "matchedRules": ["MF_FOOD_PRIV_KR_IN_001"],
  "categories": ["pii.employee_contact"],
  "normalizedText": "포장라인 반장 홍길동(01012345678)에게 연락해",
  "redactedText": "포장라인 반장 [PERSON](010-****-****)에게 연락해",
  "message": "직원 연락처 등 개인정보는 입력할 수 없습니다.",
```

```
  "policyVersion": "2026.05.1",
  "decisionId": "dec_xyz789",
  "evaluatedDomains": ["global", "manufacturing", "manufacturing-
        food"]
}
```

6.2 `validate_output`

LLM 응답에 대한 검증. 입출력 스키마는 `validate_input`과 동일하되 `ioType=output` 정책만 평가.

6.3 `sanitize_input` / `redact_output`

판정 없이 마스킹된 결과만 반환 (RAG 인덱싱 전처리 등).

6.4 `classify_privacy`

PII 카테고리 분류만 수행 (action 없음, 분석용).

6.5 `get_policy_bundle`

```
{
  "domain": "manufacturing-food",
  "locale": "ko-KR",
  "ioType": "input",
  "version": "latest"
}
```

활성 정책 목록 반환 (디버깅용).

6.6 `explain_decision`

```
{ "decisionId": "dec_xyz789" }
```

판정 근거 상세 반환 (어떤 룰이 매칭, 어떤 패턴, 점수 분해).

6.7 `health_check`

서버 상태, DB 연결, 정책 로드 상태 반환.

---

7. 소프트웨어 스택

7.1 기술 스택 결정

컴포넌트	스택	이유
MCP Server		

컴포넌트	스택	이유
	Node + TypeScript + **NestJS**	MCP SDK(TS) 친화, NestJS DI/모듈/Guard 구조 적합
Policy Engine	NestJS	MCP 서버와 monorepo 공유, REST API 노출
Admin Console	**Next.js** (App Router) + TypeScript	정책·감사·도메인 관리 UI
Detection Engine	**Python** + FastAPI	spaCy, Presidio, transformers 생태계
Internal Tooling	**Vite** + React + TS	개발용 정책 시뮬레이터, 룰 작성 도구
공통 라이브러리	TypeScript (zod 스키마)	클라이언트 SDK도 공유
Database	**PostgreSQL** 16+	JSONB 정책 저장, 감사 로그
Cache	**Redis** 7+	정책 캐시, rate limit, 세션
Message Queue	Redis Streams (또는 BullMQ)	감사 비동기 적재
Observability	OpenTelemetry + Prometheus + Grafana	
Container	Docker + Docker Compose / Kubernetes	

7.2 모노레포 구조 (제안)

```
mcp-guardrails-service/
├── apps/
│   ├── mcp-server/          # NestJS, MCP stdio + HTTP/SSE
│   ├── policy-api/          # NestJS REST API
│   ├── admin-console/       # Next.js
│   ├── detection-engine/    # Python FastAPI
│   └── rule-studio/         # Vite + React (정책 작성 IDE)
├── packages/
│   ├── shared-types/        # zod schemas, TS types
│   ├── policy-schema/       # JSON schema 정의
│   ├── client-sdk-node/     # MCP client SDK
│   └── client-sdk-python/   # Python SDK
├── policies/                # JSON 정책 번들 (Git 관리)
├── tests/
│   ├── unit/
│   ├── integration/
│   └── e2e/
├── docker/
├── k8s/
└── docs/
```

7.3 데이터베이스 스키마 (핵심 테이블)

```sql
-- 테넌트
CREATE TABLE tenants (
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 도메인 레지스트리
CREATE TABLE domains (
  domain_key TEXT PRIMARY KEY,          -- "manufacturing-food"
  parent_key TEXT REFERENCES domains(domain_key),
  display_name JSONB,
  default_locales TEXT[],
  regulation_tags TEXT[],
  status TEXT CHECK (status IN ('active','beta','deprecated')),
  version TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 정책 룰
CREATE TABLE rules (
  rule_id TEXT PRIMARY KEY,
  domain TEXT REFERENCES domains(domain_key),
  locale TEXT NOT NULL,
  io_type TEXT CHECK (io_type IN ('input','output','both')),
  category TEXT,
  sub_category TEXT,
  severity TEXT,
  priority INT,
  action TEXT,
  pattern_bundle JSONB,
  message_template JSONB,
  regulation_tags TEXT[],
  examples_positive TEXT[],
  examples_negative TEXT[],
  enabled BOOLEAN DEFAULT true,
  version TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_rules_domain_locale_io ON rules(domain, locale,
        io_type) WHERE enabled = true;

-- 정책 버전
CREATE TABLE policy_versions (
  id UUID PRIMARY KEY,
  domain TEXT NOT NULL,
  version TEXT NOT NULL,
  bundle JSONB,
  status TEXT CHECK (status IN
       ('draft','staged','active','archived')),
```

```
  created_by UUID,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Whitelist
CREATE TABLE whitelists (
  id UUID PRIMARY KEY,
  domain TEXT,
  locale TEXT,
  category TEXT,
  patterns TEXT[],
  score_adjustment NUMERIC
);

-- 감사 로그
CREATE TABLE decision_events (
  decision_id UUID PRIMARY KEY,
  tenant_id UUID REFERENCES tenants(id),
  session_id TEXT,
  domain TEXT,
  locale TEXT,
  io_type TEXT,
  text_hash TEXT,           -- SHA-256
  text_encrypted BYTEA,     -- optional, AES-256
  decision TEXT,
  risk_score NUMERIC,
  matched_rules TEXT[],
  policy_version TEXT,
  latency_ms INT,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_audit_tenant_time ON decision_events(tenant_id,
        created_at DESC);

-- API 키
CREATE TABLE api_keys (
  id UUID PRIMARY KEY,
  tenant_id UUID,
  key_hash TEXT UNIQUE,
  scopes TEXT[],           --
       ["validate_input","validate_output",...]
  allowed_domains TEXT[],
  expires_at TIMESTAMPTZ
);

-- 테스트 케이스
CREATE TABLE test_cases (
  id UUID PRIMARY KEY,
  domain TEXT,
  locale TEXT,
  io_type TEXT,
  input_text TEXT,
  expected_decision TEXT,
```

```
  expected_rule_ids TEXT[],
  tags TEXT[]
);
```

7.4 Redis 키 설계

```
bundle:{domain}:{locale}:{io_type}              -> compiled rule
list (JSON, TTL 5min)
ratelimit:{api_key}:{minute}                    -> counter
session:{session_id}:risk                       -> rolling risk
score
lock:policy_reload:{domain}                     -> distributed
lock
cache:detect:{sha256(text)}:{domain}:{locale}   -> 동일 입력 결과 캐시
(TTL 60s)
```

---

8. 처리 파이프라인 상세

8.1 Input 검증 흐름

```
1. MCP tool call 수신 (validate_input)
2. API Key 인증 → tenant 식별
3. Rate limit 체크 (Redis)
4. 입력 캐시 조회 (sha256 → cache:detect:...)
   └ hit: 즉시 반환
5. 정규화 (Detection Engine 호출)
   - NFKC, 공백 통일, 토큰 분리, 숫자 정규화
6. 정책 번들 로드 (Redis → 없으면 DB)
   - domain 머지 순서 적용: global → manufacturing → manufacturing-
food
7. Fast path: regex/keyword 매칭
8. Slow path: NER + classifier (필요 시)
9. Whitelist 보정 (점수 감산)
10. Decision aggregation
    - 매칭된 룰들의 severity/priority/score 기반
    - 가장 엄격한 action 채택
11. Audit log (비동기, Redis Stream → PostgreSQL)
12. 응답 캐시 저장
13. MCP 응답 반환
```

8.2 ko-KR/en/ja-JP 혼합 입력 정규화

1. Unicode NFKC
2. 공백·구분자 정규화 (다중 공백→1, 탭/줄바꿈 정리)
3. 스크립트 단위 토큰화 (Hangul/Latin/CJK/Digit/Symbol)
4. 숫자 패턴 전처리 (공백·하이픈 제거 후 패턴 매칭)
5. 대소문자 정규화 (case-fold)
6. 로마자 변환 (옵션)

예: `John(010-1234-5678)`이 `line 3`에서 `issue` 신고 → 전화번호 패턴 탐지 → `John([PHONE])`이 `line 3`에서 `issue` 신고

---

9. 보안 요구사항

항목	요구사항
인증	API Key + OAuth 2.0 (Client Credentials)
권한	RBAC: tenant × domain × tool scope
전송 보안	TLS 1.3 (HTTP/SSE), stdio는 OS 권한 의존
저장 보안	DB at-rest 암호화, 원문은 해시 우선
시크릿 관리	Vault/AWS Secrets Manager
감사	모든 판정·정책 변경 immutable log
테넌트 격리	row-level security (PostgreSQL RLS)
정책 무결성	정책 파일 서명(GPG/sigstore)
Rate Limit	per API key, per tool, per tenant
PII 최소 보관	원문은 옵션 + TTL 90일 자동 삭제
Field-level redaction	역할별 redaction 정책 차등

---

10. 성능·확장성 요구사항

지표	목표
`validate_input` P95 latency	< 100ms (cached), < 300ms (uncached)
`validate_output` P95 latency	< 200ms (cached), < 500ms (uncached)
처리량	단일 인스턴스 1,000 RPS
정책 hot reload	< 5초 무중단
가용성	99.9% (월 43분 down)
정책 수 확장	도메인당 10,000 룰까지
동시 도메인 로드	단일 인스턴스에서 50 도메인
캐시 적중률	> 60%

스케일 전략: - MCP Server / Policy API: stateless, horizontal scale - Detection Engine: GPU 옵션, batch 처리 - PostgreSQL: read replica + connection pool (pgBouncer) - Redis: cluster mode

---

11. MCP 클라이언트 통합 가이드

11.1 Claude Desktop

설정 파일: - macOS: `~/Library/Application Support/Claude/claude_desktop_config.json` - Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```
{
  "mcpServers": {
    "guardrails": {
      "command": "npx",
      "args": ["-y", "@your-org/guardrails-mcp"],
      "env": {
        "GUARDRAILS_ENDPOINT": "https://guardrails.example.com",
        "GUARDRAILS_API_KEY": "key_xxx",
        "GUARDRAILS_DOMAINS": "manufacturing-food,manufacturing-
        chemical",
        "GUARDRAILS_LOCALES": "ko-KR,en"
      }
    }
  }
}
```

11.2 Gemini CLI

`~/.gemini/settings.json`:

```
{
  "mcpServers": {
    "guardrails": {
      "command": "node",
      "args": ["/opt/guardrails/dist/server.js"],
      "env": {
        "GUARDRAILS_DOMAINS": "manufacturing-food",
        "GUARDRAILS_LOCALES": "ko-KR"
      }
    }
  }
}
```

11.3 Cursor / Continue / Cline

각 클라이언트의 MCP 설정에 동일한 형태로 등록.

11.4 원격 SSE 연결

```
{
  "mcpServers": {
    "guardrails-remote": {
```

```
      "command": "npx",
      "args": ["mcp-remote", "https://guardrails.example.com/
        sse"],
      "env": {
        "GUARDRAILS_API_KEY": "key_xxx"
      }
    }
  }
}
```

11.5 권장 시스템 프롬프트

```
You MUST call the `guardrails.validate_input` tool before
processing any user input that may contain personal data, worker
identity, supplier contacts, consumer complaints, health
information, financial credentials, or compliance-sensitive
content.

You MUST call `guardrails.validate_output` before delivering any
generated report, CSV, table, or summary to the user.

Rules:
- If `decision == "block"`, do not proceed. Show the `message`
field to the user.
- If `decision == "redact"` or `"sanitize"`, only use the
`redactedText` field downstream.
- If `decision == "escalate"`, deliver only a generic
acknowledgement and notify the operator.
- Active domains for this session: ${GUARDRAILS_DOMAINS}.
- Active locales: ${GUARDRAILS_LOCALES}.
```

---

12. Admin Console (Next.js) 기능

화면	기능
Dashboard	판정 건수, decision 분포, 도메인별 적중률, latency
Domains	도메인 등록·상속·상태
Rules	CRUD, JSON 편집기, 시뮬레이션
Bundles	버전 관리, draft/staged/active 승격
Whitelist	예외 표현 관리
Test Cases	테스트 케이스 작성·회귀 실행
Audit	감사 로그 검색·필터 (decision, rule, time, tenant)
Tenants	테넌트·API Key 관리, allowed_domains 설정
Settings	시스템 환경, Redis/DB 상태, hot reload

---

13. 테스트 전략

13.1 테스트 계층

- **Unit**: 정규식, 정규화 함수, score aggregation
- **Integration**: Detection Engine ? Policy Engine ? Redis/PG
- **Contract**: MCP tool 입출력 스키마 (zod 기반)
- **E2E**: Claude Desktop / Gemini CLI 실제 연결 시나리오
- **Regression**: 도메인별 테스트 케이스 자동 실행 (CI)
- **Adversarial**: 우회 입력 (공백 삽입, base64, 분할), prompt injection
- **Performance**: k6/Locust 부하 테스트

13.2 도메인별 테스트 케이스 최소치

도메인당 input 50 + output 50 = 100 케이스. 일반 케이스 / edge case / 우회 케이스 비율 6:3:1.

---

14. 운영 및 관측성

- 메트릭: latency, decision 분포, cache hit, rule hit, error rate
- 로깅: structured JSON, 원문 마스킹
- 트레이싱: OpenTelemetry, MCP tool → Detection → DB span
- 알림: high-risk burst, 정책 reload 실패, DB lag
- 정책 변경 감지: webhook → Slack/Teams

---

15. 마일스톤

단계	기간	산출물
M1 PoC	4주	MCP Server(stdio) + Detection(Python) + manufacturing-food ko-KR/en 40개 룰, Claude Desktop 연동
M2 정책 엔진	4주	PostgreSQL/Redis 통합, JSON hot reload, Whitelist, Test Cases 자동화
M3 Admin Console	4주	Next.js 콘솔, 도메인·룰·번들 CRUD, 감사 조회
M4 중앙 서비스화	4주	HTTP/SSE transport, OAuth, RBAC, k8s 배포
M5 도메인 확장	6주	finance-retail-banking, healthcare-clinical, manufacturing-chemical 추가

단계	기간	산출물
M6 RAG 결합	4주	retrieval 단계 sanitize, prompt injection 강화
M7 GA	2주	SLA·문서·SDK 공개

16. 리스크 및 완화책

리스크	영향	완화책
과도한 차단(오탐)	UX 저하	Whitelist + 다단계 임계값 + Audit 기반 튜닝
누락(미탐)	규제·보안 사고	Recall 우선 임계값, adversarial 테스트, 다단계 필터
정책 충돌 (다중 도메인)	예측 불가 응답	머지 우선순위 명문화, 시뮬레이터 제공
정책 hot reload 장애	서비스 영향	카나리 reload, 이전 버전 즉시 롤백
MCP 클라이언트가 tool 호출 누락	가드레일 우회	시스템 프롬프트 강제, 클라이언트 SDK wrapper 권장
RAG context로 가드레일 flip	안전판단 불안정	RAG 전용 입력 전처리, context-stripping 모드
원문 로그 PII 누출	규제 위반	해시·암호화 저장, RBAC, TTL 자동 삭제
도메인 수 폭증 시 캐시 메모리	비용 증가	LRU eviction, bundle compaction

17. 오픈 이슈

1. 일부 MCP 클라이언트(특히 데스크톱 앱)에서 환경변수 배열 전달 방식 차이 → CSV string으로 통일 (`"manufacturing-food,manufacturing-chemical"`).
2. ja-JP 형태소 분석기 선정 (MeCab vs Sudachi) — Detection Engine 초기 구현 시 결정.
3. LLM-as-judge 사용 시 외부 API 비용/지연 vs 내부 소형 모델 — M4 단계에서 벤치마크.
4. 정책 라이선스/저작권 — 외부 가이드(OWASP, NIST) 인용 시 license 검증.
5. 다국어 메시지 템플릿 i18n 관리 — Crowdin/Lokalise 도입 여부.

18. 도메인 확장 로드맵 (ISIC/KSIC/NACE/NAICS 기반)

본 서비스의 도메인 키는 **ISIC Rev.4 Section/Division** 구조와 1:1 매핑되며, 동일 노드에 KSIC·NACE·NAICS 코드를 함께 태깅한다. 단계적 확장은 (a) 산업 분류상 인접성, (b) 규제 압력(개인정보·안전), (c) 시장 수요 우선순위 기준으로 정한다.

참고: `.doc/Perplexity-ISIC,KSIC,NACE,NAICS 분류 조사 및 정리.md`.

18.1 도메인 키 — 산업분류 매핑 원칙

domain_key	ISIC Rev.4	KSIC	NACE Rev.2	NAICS
global	–	–	–	
manufacturing	Section C	C	C	31–33
manufacturing-food	10	10	10	311
manufacturing-beverage	11	11	11	312
manufacturing-tobacco	12	12	12	3122
manufacturing-textile	13	13	13	313–314
manufacturing-apparel	14	14	14	315
manufacturing-chemical	20	20	20	325
manufacturing-pharma	21	21	21	3254
manufacturing-rubber-plastics	22	22	22	326
manufacturing-metal-primary	24	24	24	331
manufacturing-electronics	26	26	26	334
manufacturing-electrical	27	28	27	335
manufacturing-machinery	28	29	28	333
manufacturing-automotive	29	30	29	3361–3363
manufacturing-other-transport	30	31	30	3364–3369
finance-banking	64	64	64	522
finance-insurance	65	65	65	524

```
finance-securities-misc             66          66
66          523
healthcare-clinical                 86          86
86          621-622
healthcare-residential-welfare      87          87
87          623
healthcare-non-residential-welfare  88          88
88          624
public-administration               84          84
84          92
ict-software                        62          62
62          5415
ict-info-services                   63          63
63          519
retail-wholesale                    46          46
46          42
retail-stores                       47          47
47          44-45
transport-logistics                 49-53       49-53
49-53       48-49
energy-utilities                    35          35
35          22
construction                        41-43       41-43
41-43       23
education                           85          85
85          61
arts-recreation                     90-93       90-92
90-93       71
agriculture                         01-03       01-03
01-03       11
```

명명 규칙(PRD §4.1.2): `<industry>-<sub_industry>[-<function>]`. 더 세분화 필요시 `<industry>-<sub>-<class>` 추가(예: `manufacturing-automotive-battery`).

18.2 단계별 확장 로드맵

각 phase는 (a) 도메인 키, (b) ISIC/KSIC 매핑, (c) 적용 규제, (d) 시드 룰 목표 수, (e) 합쳐 배포 가능한 phase 조합을 명시한다. Phase 간 독립 배포 보장 (이전 phase 결과물 단독 구동 가능, `docker-compose.<phase>.yml` overlay 추가만으로 확장).

Phase 1 — `global` (Privacy & Common Safety) ✅

- ISIC: 모든 Section에 공통 적용 (cross-cutting)
- 규제: KR_PIPA, EU_GDPR (Art.5 / Art.9 / Art.10), US_HIPAA, US_CCPA, JP_APPI, COPPA, PCI_DSS
- 시드 룰: **211개** (input/output 각각, ko-KR / en) — `scripts/privacy.{kr,en}.json`
- 카테고리: pii / financial / credentials / health / location / biometrics / sensitive / children / scope / compliance / hallucination
- 배포: `docker-compose.yml + docker-compose.privacy.yml`

- 상태: **GA (Phase 1 완료)**

Phase 2 — `manufacturing-food` (KSIC 10 / ISIC 10) ⍰

- ISIC C10, KSIC 10, NACE 10, NAICS 311
- 규제: KR_FSC(식품위생법), 식품 표시광고법, KR_OSH, US_FDA(FALCPA/HACCP), EU_FOOD, TRADE_SECRET
- 시드 룰: **53개** 초안 — `scripts/manufacturing-food.{kr,en}.json` (pii=23, compliance=14, ip=10, safety=6)
- 추가 작업 (M5):
  - 도메인 특화 100+ 룰까지 확장 (식품위생법 세조항 매핑)
  - whitelist (식품 일반 용어, 법령 인용)
  - 테스트 케이스 100+ (정상/edge/우회 6:3:1)
- 배포 조합:
  - 단독: `docker-compose.yml + docker-compose.privacy.yml + docker-compose.food.yml`
  - 머지 순서: `global → manufacturing → manufacturing-food`
- 상태: **개발 중 (1차 초안)**

Phase 3 — `manufacturing-chemical` (KSIC 20 / ISIC 20)

- ISIC C20, KSIC 20, NACE 20, NAICS 325
- 규제: 화학물질관리법, REACH, US_TSCA, MSDS, OSHA HazCom, KR_OSH
- 시드 룰 목표: ~120개
- 핵심 sub_category:
  - 위험물 운반·보관 SOP 우회 차단
  - MSDS 변조·은폐 차단
  - 환경 배출 보고 위·변조 차단
  - 공정 비밀(촉매 비율, 반응 조건) 보호
  - 임직원 노출 데이터(혈중 농도) PII 보호
- 배포 조합: P1 + P2 + P3 또는 P1 + P3 (식품 안 쓰는 조직)

Phase 4 — `manufacturing-pharma` (KSIC 21 / ISIC 21)

- ISIC C21, KSIC 21, NACE 21, NAICS 3254
- 규제: 약사법, GMP, US_FDA(21 CFR Part 11), EU_EMA, ICH-GCP, US_HIPAA(임상)
- 시드 룰 목표: ~140개
- 핵심 sub_category:
  - 임상시험 피험자 식별(IND/CTA) 보호
  - 적응증 외 사용(off-label) 권유 차단
  - 부작용·SAE 누락 보고 차단
  - 의약품 광고 효능 과장 차단
  - 제조처방·공정 비밀 보호 (rare disease drug 등)

Phase 5 — `manufacturing-automotive` (KSIC 30 / ISIC 29)

- ISIC C29, KSIC 30, NACE 29, NAICS 3361-3363
- 규제: 자동차관리법, 리콜 규정, UNECE WP.29, ISO 26262(Functional Safety), SOTIF
- 시드 룰 목표: ~110개

- 핵심 sub_category:
  - 안전 결함 은폐 권유 차단
  - 배출가스 조작 권유 차단
  - 자율주행 책임 회피 권유 차단
  - 협력사 도면·BOM 비밀 보호

Phase 6 — `manufacturing-electronics` (KSIC 26 / ISIC 26)

- ISIC C26, KSIC 26, NACE 26, NAICS 334
- 규제: 전기용품안전관리법, KR_RoHS, EU_RoHS, EU_CE, US_FCC, export control(EAR/ITAR)
- 시드 룰 목표: ~100개
- 핵심 sub_category:
  - 반도체 공정 레시피 비밀 보호
  - 수출통제 품목 우회 차단
  - 부품 위조·인증 위반 차단

Phase 7 — `finance-banking` / `finance-insurance` / `finance-securities` (KSIC 64-66)

- ISIC K, KSIC K, NACE K, NAICS 52
- 규제: 자본시장법, 금융소비자보호법, 전자금융거래법, FATCA, MiFID II, GLBA, AML/KYC
- 시드 룰 목표: 도메인 합산 ~250개
- 핵심 sub_category:
  - 투자 권유·확정 수익 표현 차단
  - AML/KYC 우회 권유 차단
  - 시세조종·내부자거래 권유 차단
  - 보험 약관 설명의무 우회 차단

Phase 8 — `healthcare-clinical` (KSIC 86 / ISIC 86)

- ISIC Q86, KSIC 86, NACE 86, NAICS 621-622
- 규제: 의료법, US_HIPAA, EU_GDPR Art.9, 원격의료 규제, FDA SaMD
- 시드 룰 목표: ~180개
- 핵심 sub_category:
  - 진단·처방 직접 안내 차단
  - 자살·자해 정보 차단 + 지원 채널 안내
  - PHI/EMR 보호
  - 임상 가이드라인 환각 차단

Phase 9 — `public-administration` (KSIC 84 / ISIC 84)

- ISIC O84, KSIC 84, NACE 84, NAICS 92
- 규제: 행정절차법, 정보공개법, 개인정보보호법, 청탁금지법
- 시드 룰 목표: ~120개

Phase 10+ — 추가 도메인

- `ict-software` (KSIC 62), `ict-info-services` (KSIC 63)
- `retail-stores` (KSIC 47), `retail-wholesale` (KSIC 46)
- `transport-logistics` (KSIC H), `energy-utilities` (KSIC 35)

- `education` (KSIC 85): US_FERPA, KR_PIPA(학생)
- `construction` (KSIC F), `agriculture` (KSIC A)

18.3 Phase 조합 배포 매트릭스

도메인 키는 직교(orthogonal)하므로, 한 조직이 운영하는 사업 portfolio에 따라 phase를 조합 배포한다. Compose overlay 1장씩 추가하면 즉시 활성화.

조직 유형	Phase 조합	환경 변수
식품 제조 중소기업	P1 + P2	`GUARDRAILS_ENABLED_DOMAINS=global,manufacturing-food`
식품 + 화학 복합 그룹	P1 + P2 + P3	`global,manufacturing-food,manufacturing-chemical`
제약 기업	P1 + P4	`global,manufacturing-pharma`
종합 제조 (자동차+전자)	P1 + P5 + P6	`global,manufacturing-automotive,manufacturing-electronics`
은행 + 증권 + 보험	P1 + P7(3종)	`global,finance-banking,finance-insurance,finance-securities-misc`
종합 병원 + 보건소	P1 + P8	`global,healthcare-clinical`
핀테크 (은행 +IT)	P1 + P7 + P10(ict)	`global,finance-banking,ict-software`
공공 기관	P1 + P9	`global,public-administration`

Phase 머지 충돌 해소 (PRD §4.1.4): 더 엄격한 action 우선 (block > redact > warn > allow), 동률 시 priority 큰 쪽 + 더 specific 도메인 우선.

18.4 도메인 신규 추가 / 회수 절차

신규 추가: 1. ISIC/KSIC division 결정 → domain_key 부여 2. `policies/<domain_key>/{privacy,compliance,safety,ip,...}/{input,output}.{locale}.json` 시드 작성 (최소 50 rules) 3. `whitelists/<domain_key>.<locale>.json` 예외 표현 정의 4. `tests/<domain_key>/` 회귀

테스트 100+ 케이스 5. Detection Engine `src/detectors/<domain_module>.py` 도메인 사전 추가 (필요시) 6. Compose overlay `docker-compose.<domain>.yml` 추가 (`GUARDRAILS_ENABLED_DOMAINS` 머지) 7. Staging 회귀 → 정책 버전 `active` 승격 → 캐시 무효화

회수(deprecation): - domain 레지스트리 status `deprecated`로 변경 - compose overlay 제거 - 정책 버전은 `archived` 보존, 신규 평가 시 머지에서 제외

---

19. 참고

- `.doc/Perplexity-Guardrails for LLM,RAG, 도메인별.md` — 도메인 분류, Privacy 240+ 룰, 식품 제조 도메인 40룰, MCP 통합 방안 원본
- `.doc/Perplexity-ISIC,KSIC,NACE,NAICS 분류 조사 및 정리.md` — 산업분류 체계 비교, 제조업 division 매핑
- `scripts/privacy.{kr,en}.json` — Phase 1 시드 룰 (각 211개)
- `scripts/manufacturing-food.{kr,en}.json` — Phase 2 시드 룰 (각 53개)
- NVIDIA NeMo Guardrails, Guardrails AI, MS Presidio
- OWASP Top 10 for LLM Applications
- GDPR Art.5/Art.9, KR PIPA, HIPAA, COPPA, PCI_DSS
- Model Context Protocol Specification (modelcontextprotocol.io)